

Day 2 AI Engineering Challenges

Build small AI workflows using memory, structured outputs, validation, and system payloads

Participant Handout

AI Engineer Mindset

An AI engineer does not stop when the model replies. An AI engineer designs what happens after the model replies.

Your workflow should follow this pattern:

Messy input -> LLM extraction -> structured JSON -> validation -> system payload -> reviewable output

What to submit through GitHub

- Fork the workshop repository.
- Create a folder named "day-off" inside the day-2 folder.
- Create your scripts in it
- Commit your changes to your fork.
- Push your branch.
- Open a Pull Request to the original repository for review.

Pull Request title format

Day 2 Challenge - <Your Name> - <Challenge Name>

Challenge 1: AI Bug Triage Workflow

Scenario

Your QA team receives messy bug reports from testers. The reports are inconsistent, incomplete, and written in natural language. Your job is to build an AI workflow that converts a messy bug report into a structured triage object and prepares it for downstream systems.

Your workflow should do the following

- Accept a messy bug report as input.
- Use the LLM to extract a structured JSON object.
- Validate whether the required fields are present.
- Detect missing information instead of guessing blindly.
- Decide whether the bug is a release blocker.
- Generate a Jira-style payload from the structured JSON.
- Generate a Slack-style alert if the bug is a release blocker.

Starter input

During regression testing on staging, the login page is not working properly in Chrome.
I entered a valid username and password, clicked the Login button, and the page just kept loading.
Expected result: user should land on the dashboard.
Actual result: loading spinner stays forever.
This is blocking our release testing today. Firefox seems to work fine.

Required structured JSON shape

```
{
  "title": "",
  "module": "",
  "environment": "",
```

```

    "browser": "",
    "severity": "",
    "release_blocker": false,
    "steps_to_reproduce": [],
    "expected_result": "",
    "actual_result": "",
    "missing_information": [],
    "suggested_next_action": ""
  }
}

```

Validation requirements

- The output must be valid JSON.
- `steps_to_reproduce` must be a list.
- `release_blocker` must be a boolean.
- `severity` should be one of: low, medium, high, critical.
- `missing_information` should list unclear or missing details.
- The workflow should not crash if the AI returns invalid JSON.

Jira-style payload to generate

```

{
  "summary": "Login page stuck after valid login on Chrome",
  "priority": "high",
  "labels": ["authentication", "Chrome", "staging"],
  "description": "Steps, expected result, actual result, and suggested next action"
}

```

Slack-style alert condition

Generate a Slack-style alert only if `release_blocker` is true.

Release Blocker Detected

Bug: Login page stuck after valid login on Chrome
 Module: authentication
 Severity: high
 Environment: staging

Suggested action:
 Assign to authentication team and collect console logs.

Expected files

- `structured_bug_workflow.py`
- `README.md` explaining your workflow, validation, and what could go wrong

Challenge 2: AI Test Case Generator Workflow

Scenario

Your product team gives you a messy requirement. It is not written in perfect test-case format. Your job is to build an AI workflow that converts the requirement into structured test cases and prepares a test-management-style payload.

Your workflow should do the following

- Accept a product requirement as input.
- Extract the feature name.
- Generate positive, negative, and edge test cases.
- Return valid structured JSON.
- Validate that each test case has required fields.
- Generate a test-management-style payload.

- Generate a coverage summary.

Starter input

Users should be able to reset their password using an OTP sent to their registered email.
 The OTP should expire after 10 minutes.
 If the user enters the wrong OTP three times, the account should be temporarily locked.

Required structured JSON shape

```
{
  "feature": "Password reset using OTP",
  "test_cases": [
    {
      "id": "TC001",
      "title": "",
      "type": "positive",
      "preconditions": [],
      "steps": [],
      "expected_result": "",
      "priority": "medium"
    }
  ],
  "edge_cases": [],
  "missing_requirements": []
}
```

Validation requirements

- The output must be valid JSON.
- test_cases must be a list.
- Each test case must include id, title, type, steps, expected_result, and priority.
- type should be one of: positive, negative, edge.
- steps must be a list of clear actions.
- missing_requirements should capture unclear requirements or assumptions.

Test-management-style payload to generate

```
{
  "suite_name": "Password reset using OTP",
  "test_cases": [
    {
      "title": "Verify password reset with valid OTP",
      "priority": "high",
      "steps": [],
      "expected_result": ""
    }
  ]
}
```

Coverage summary to generate

```
{
  "positive_cases": 2,
  "negative_cases": 3,
  "edge_cases": 2,
  "missing_requirements_count": 1
}
```

Expected files

- test_case_generator_workflow.py
- README.md explaining your workflow, validation, and what could go wrong

README.md Questions

Include a short README.md with your Pull Request. Answer these questions:

- What input does your workflow accept?
- What structured output does it produce?
- What validation did you add?
- What external system could this connect to?
- What can go wrong with this workflow?
- What would you improve next?

Submission Checklist

- My script runs without syntax errors.
- The AI response is parsed into JSON.
- The workflow validates required fields.
- The workflow creates at least one system-style payload.
- My README.md explains the workflow clearly.
- I committed and pushed my changes to my fork.
- I opened a Pull Request for review.

Recommended References

Use these only when you get stuck or want to go deeper. You do not need to master all of them before Day 3.

- [OpenAI Structured Outputs Guide](#)
- [OpenAI Cookbook: Structured Outputs Intro](#)
- [OpenAI Function Calling / Tools Guide](#)
- [Python json module documentation](#)
- [JSON.org](#)